END-TERM PROJECT-01 REPORT

ON

<Car Dodging Game>

B.Tech. [Computer Science and Engineering]

Semester: 6th

Section: 23412F3

Course Title & Code: Game Programming with C++ (CSE 3545)

Submitted By:

Student Name: Amulya Shrivastava

Registration No: 2341019456

Guided By: Mr. Sougata Sheet

Academic Year: 2025 -2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

FACULTY OF ENGINEERING AND TECHNOLOGY, INSTITUTE OF TECHNICAL EDUCATION AND RESEARCH

SIKSHA 'O' ANUSANDHAN (DEEMED TO BE) UNIVERSITY

BHUBANESWAR, ODISHA, INDIA

ABSTRACT

This project presents the development of a Car Dodging Game using C++ and the SFML (Simple and Fast Multimedia Library) framework. The game is designed around a player-controlled vehicle that must avoid continuously appearing enemy cars on a moving road while achieving the highest possible score. The implementation focuses on creating an interactive and engaging gaming experience by combining smooth graphical rendering, real-time keyboard interaction, collision detection, enemy spawning, score tracking, and progressively increasing difficulty levels. The project also demonstrates the effective use of the SFML library for handling graphics, movement, audio integration, and event management within a real-time game environment.

The development of the game emphasizes the application of Object-Oriented Programming concepts such as inheritance, abstraction, encapsulation, and polymorphism through the implementation of multiple classes including Car, PlayerCar, and EnemyCar. A modular and reusable code structure was adopted to improve maintainability, scalability, and overall software efficiency. Additional gameplay features such as responsive controls, dynamic enemy generation, background movement, sound effects, and scoring mechanisms contribute to a smooth and immersive user experience. The project also highlights efficient game loop management and optimized rendering techniques required for maintaining consistent gameplay performance.

The game was successfully implemented and tested under different gameplay conditions to ensure stability, responsiveness, and proper functionality. Various debugging and performance optimization techniques were applied to minimize errors and improve the reliability of the application. The project provided practical exposure to game development using C++ and SFML while strengthening programming knowledge, logical thinking, problem-solving abilities, and software design skills. Overall, the project demonstrates how multimedia libraries and object-oriented programming principles can be effectively combined to develop an entertaining and technically efficient 2D game application.

Amulya Shrivastava

Submitted By

Mr. Sougata Sheet

Guided By

TABLE OF CONTENTS

LIST OF FIGURES

01. Introduction

Game programming combines graphics rendering, event handling, physics simulation, and user interaction into a real-time software application. C++ is widely used in game development because of its high performance, efficient memory management, and better control over system resources. These features make it suitable for developing fast and responsive gaming applications. In addition, multimedia libraries such as SFML (Simple and Fast Multimedia Library) provide support for graphics, audio, animation, and keyboard input, making the development process easier and more efficient. Game development also helps programmers improve logical thinking, problem-solving skills, and understanding of real-time software systems.

This project presents a simple 2D Car Dodger Game developed using C++ and SFML. The player controls a car moving across lanes while avoiding enemy vehicles coming from the top of the screen. The game includes smooth movement, responsive keyboard controls, collision detection, score tracking, and increasing difficulty levels to make gameplay more engaging and challenging. Enemy cars are generated dynamically, and the speed of the game gradually increases as the score improves, creating an interactive gaming experience for the player.

The project helps in understanding the practical implementation of Object-Oriented Programming concepts such as inheritance, abstraction, encapsulation, and polymorphism. Different classes like Car, PlayerCar, and EnemyCar are used to maintain modularity and code reusability. The project also improves knowledge of real-time game loops, event handling, collision management, and multimedia programming using SFML libraries. Overall, the project demonstrates the practical application of C++ in game development while enhancing software design and programming skills.

02. Problem Statement

The problem addressed in this project is to design and implement a real-time Car Dodging Game using C++ and the SFML graphics library that provides smooth gameplay, responsive controls, and efficient object management. The project focuses on developing an interactive gaming application where the player controls a car and avoids continuously spawning enemy vehicles on a moving road. The game must maintain real-time rendering, proper collision handling, dynamic enemy generation, and score updates without affecting performance or gameplay smoothness. The project also aims to demonstrate how multimedia programming and object-oriented design techniques can be combined to develop an efficient and user-friendly game application.

The game is designed to satisfy the following requirements:

- Smooth player movement using keyboard controls.
- Random enemy car spawning across different lanes.
- Collision detection between player and enemy vehicles.
- Real-time score tracking system.
- Increasing difficulty during gameplay progression.
- Efficient rendering using the SFML graphics library.
- Proper game loop management and frame-based updates.
- Use of Object-Oriented Programming concepts such as inheritance and polymorphism.
- Responsive gameplay with stable performance and smooth animations.

The program is developed using programming concepts such as conditional statements, loops, functions, classes, and event handling mechanisms. The project demonstrates how C++ can be used to create real-time multimedia applications with interactive graphics and efficient game logic. It also highlights the practical implementation of software design principles, collision management, and graphical rendering techniques. Overall, the project provides valuable experience in game development, logical problem-solving, and multimedia application programming using C++ and SFML.

03. Objectives of the Project

The primary objectives of this project are established to provide a structured approach for the planning, design, and development of the Car Dodging Game while ensuring smooth gameplay, efficient functionality, and proper implementation of game development concepts. The project combines programming logic, multimedia handling, and real-time interaction using C++ and the SFML library. It also focuses on improving understanding of software design, object-oriented programming principles, and graphics rendering techniques while enhancing problem-solving abilities, logical thinking, and programming confidence.

The objectives of this practical project are:

- To develop a functional 2D car dodging game using C++ and SFML
- To implement Object-Oriented Programming concepts such as classes, inheritance, abstraction, encapsulation, and polymorphism
- To use inheritance and polymorphism effectively through a well-structured class hierarchy
- To implement collision detection mechanisms for detecting interaction between player and enemy vehicles
- To create a real-time scoring system that updates dynamically during gameplay
- To improve programming, debugging, and logical problem-solving skills through practical implementation
- To understand game loops, frame-based updates, and real-time event handling mechanisms
- To gain practical experience using the SFML multimedia library for graphics, sound, and keyboard interaction
- To create smooth gameplay with responsive controls and efficient object management
- To understand the basics of real-time game development and multimedia application design using C++

Overall, the project is designed to provide practical exposure to game programming concepts while developing technical skills related to software development, graphics rendering, and interactive application creation.

04. Tools and Technologies Used

The following tools and technologies were used in Project 1 and Project 2:

- Programming Language: C++ (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
- Graphics and Audio Library: SFML (Simple and Fast Multimedia Library)
- Compiler: g++ (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
- Development Environment: Visual Studio Code / Terminal-Based Compilation
- Operating System: Ubuntu-24.04.3 LTS

These tools provide the required environment for rendering sprites, handling keyboard input, playing sounds, displaying HUD text, and managing the real-time game loop.

05. Methodology

The methodology followed during the development of this project includes:

1. Requirement analysis:

   Identified all essential gameplay mechanics including player movement, enemy vehicle generation, collision detection, score calculation, speed increase, difficulty selection, audio integration, and restart functionality before beginning development.

2. Class design:

   Created a base Car class along with derived PlayerCar and EnemyCar classes to implement inheritance and maintain a modular object-oriented structure.

3. Asset loading:

   Loaded all required player car, enemy car, font, texture, and sound assets from the Assets folder to support graphics rendering and audio functionality.

4. Game loop:

   Implemented the main game loop using SFML window events, delta-time updates, rendering operations, and game state transitions for smooth real-time gameplay.

5. Road rendering:

   Designed a vertical road layout containing four lanes with dashed lane markings and scrolling visuals to create a realistic driving environment.

6. Enemy spawning:

   Generated enemy vehicles randomly within different lanes and adjusted their movement speed according to the selected difficulty level.

7. Collision and scoring:

   Used SFML FloatRect collision boundaries to detect vehicle crashes and updated the score based on player survival time and successfully avoided enemy cars.

Testing:

   Tested all major gameplay features including player controls, lane restrictions, collision handling, game-over conditions, pause/resume options, restart functionality, difficulty modes, HUD display, and asset loading performance.

06. Results Analysis and Screenshots

This section describes the results obtained after implementing the C program.

The program was successfully executed and produced the expected output. It demonstrates the correct use of programming concepts and provides accurate results based on user input.

Gameplay Behaviour Verified:

- Player car responds instantly to keyboard input (Arrow) with smooth, delta-time-scaled movement.
- Enemy cars spawn randomly across three lanes, descend at randomised speeds, and are cleanly removed after passing off-screen.
- Collision detection correctly ends one life per collision; the game transitions to Game Over when all three lives are lost.
- Score increments with each successfully dodged enemy car.

Game speed increases continuously; the HUD reflects both the speed level and actual pixel-per-second value in real time

The project successfully delivers a fully playable car dodging game with engaging gameplay mechanics and smooth user interaction. It demonstrates the effective implementation of core game development concepts, resulting in an enjoyable and functional gaming experience.

Figure 1: Car Dodging Game Window (Start Window)

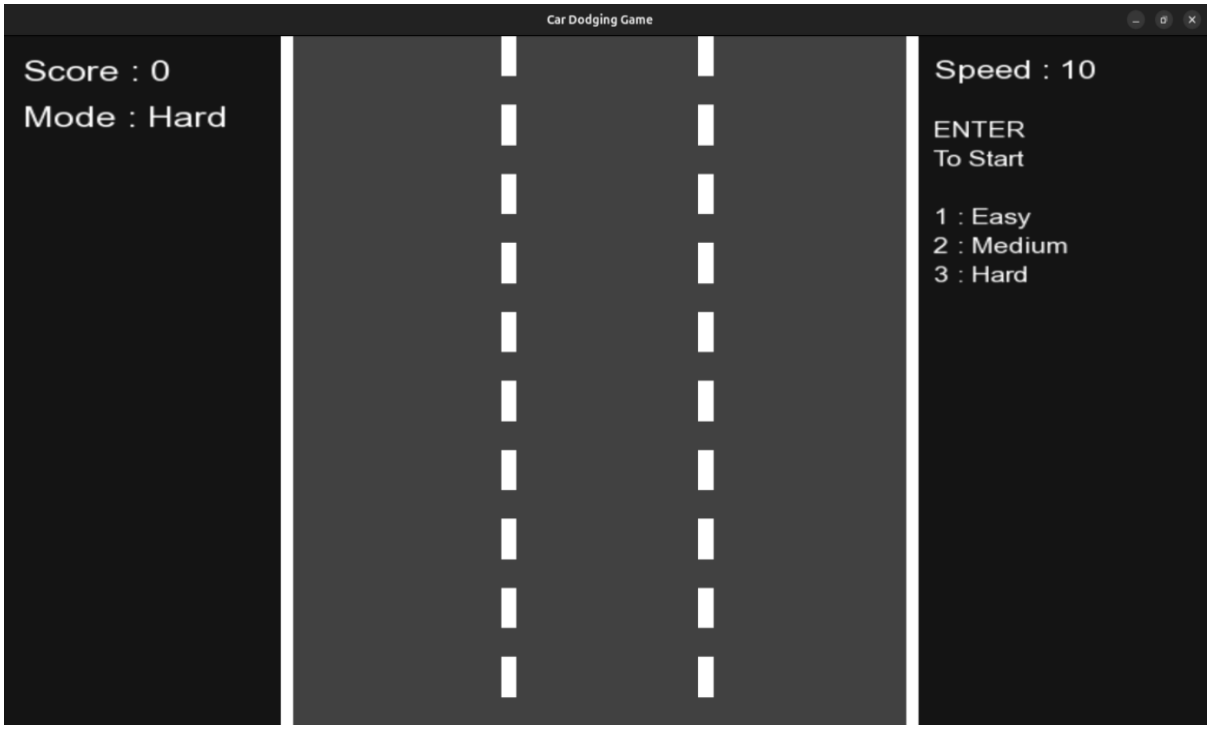


Figure 2: Medium Mode Game Play (By Default Set To Medium)

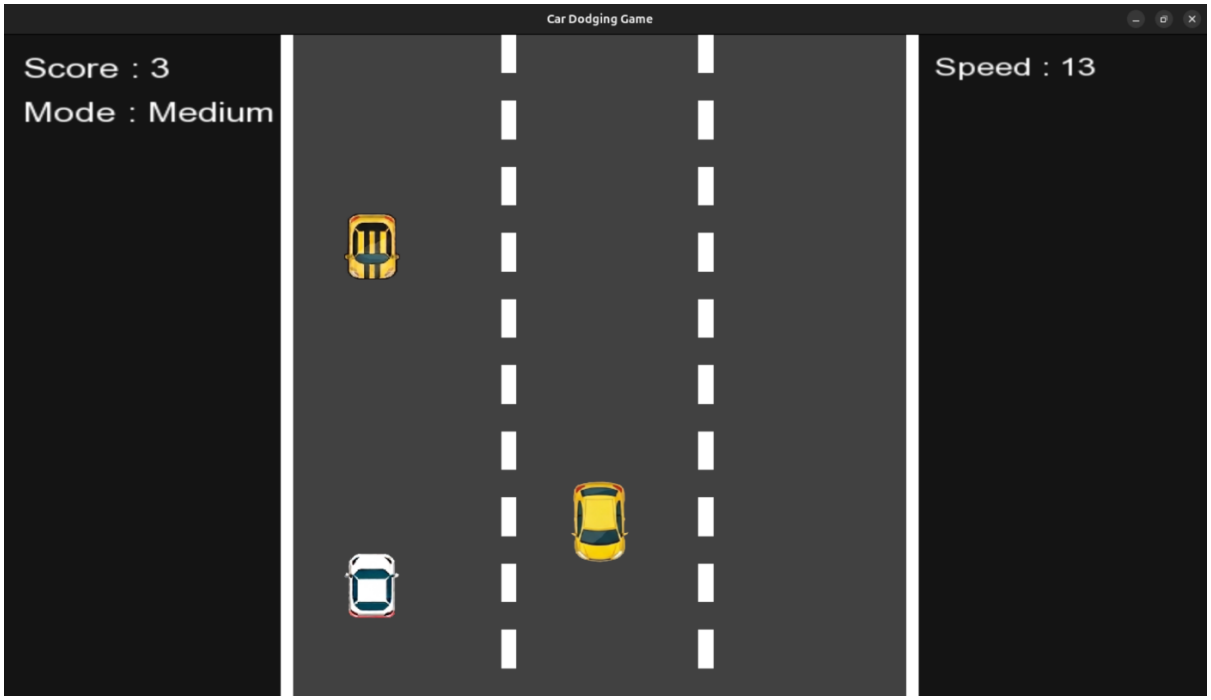


Figure 3: Hard Mode Selected After Restart (Modes Working)

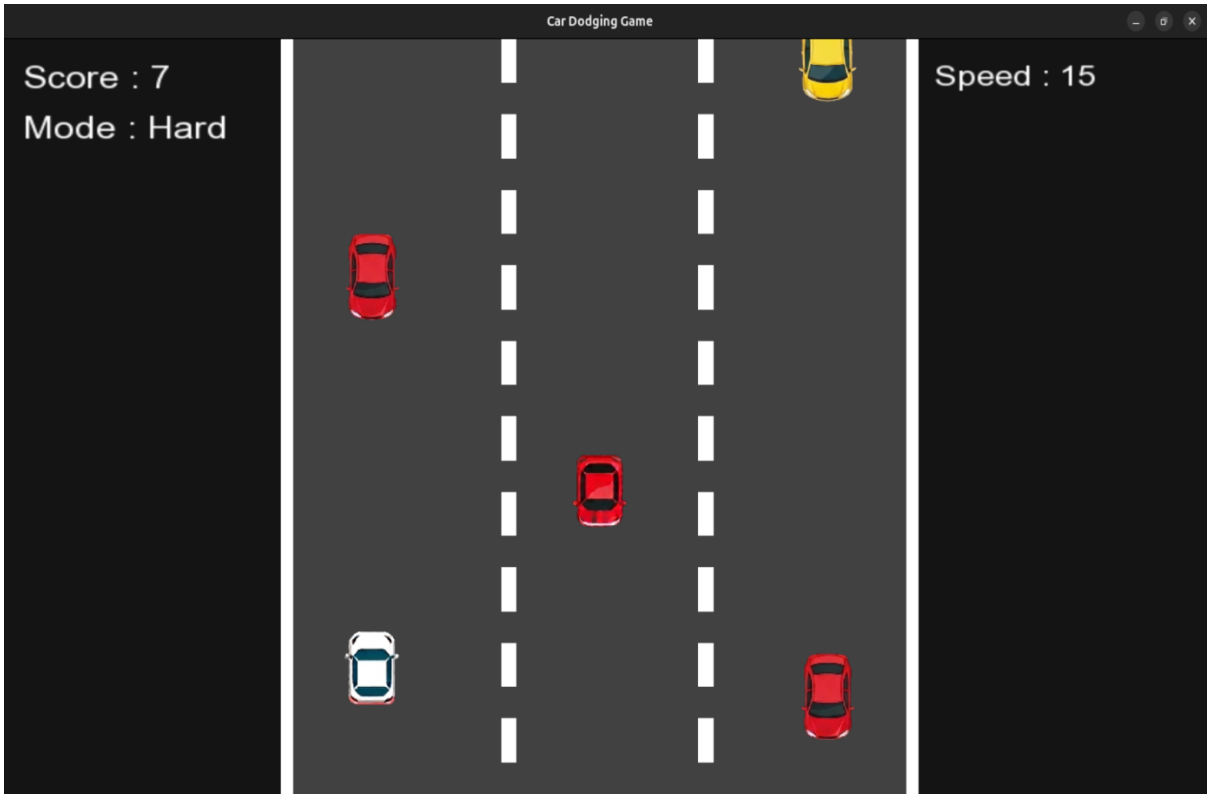


Figure 4: Game Pause and Resume Feature (With P & R Key)

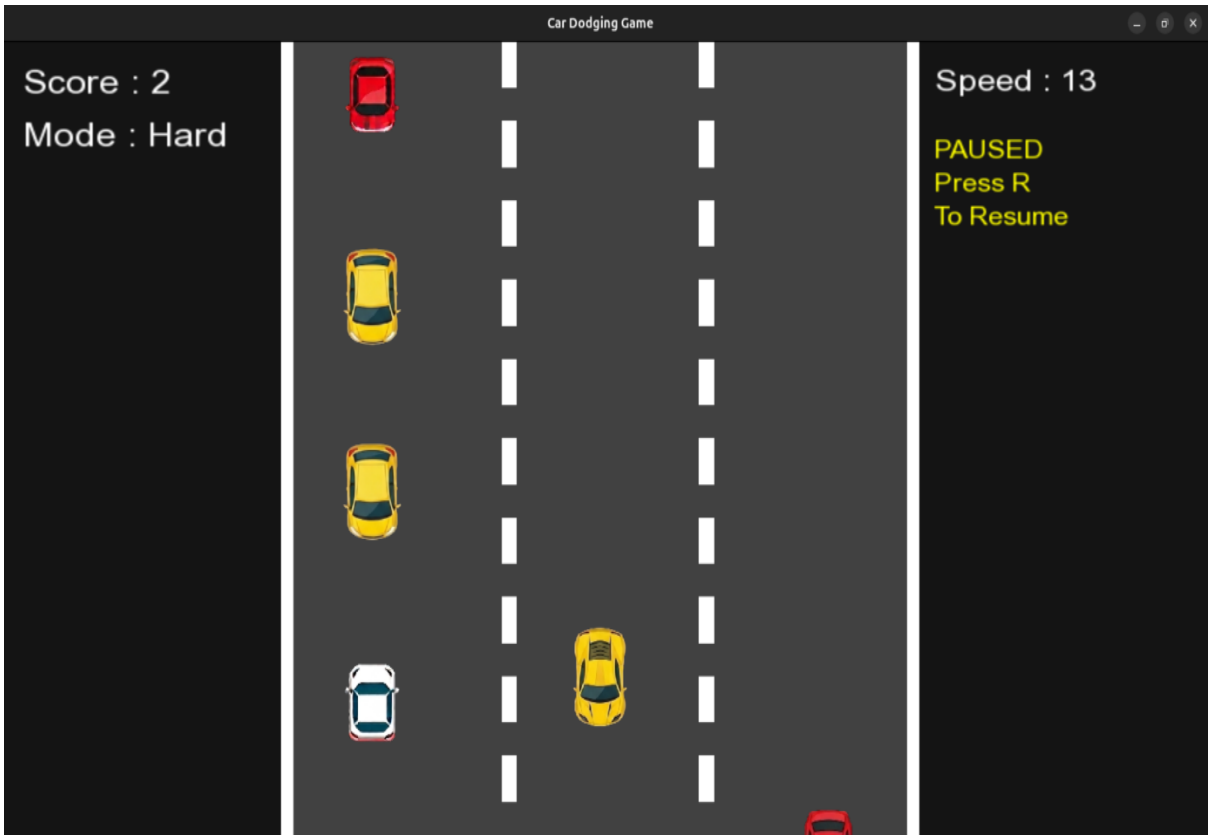
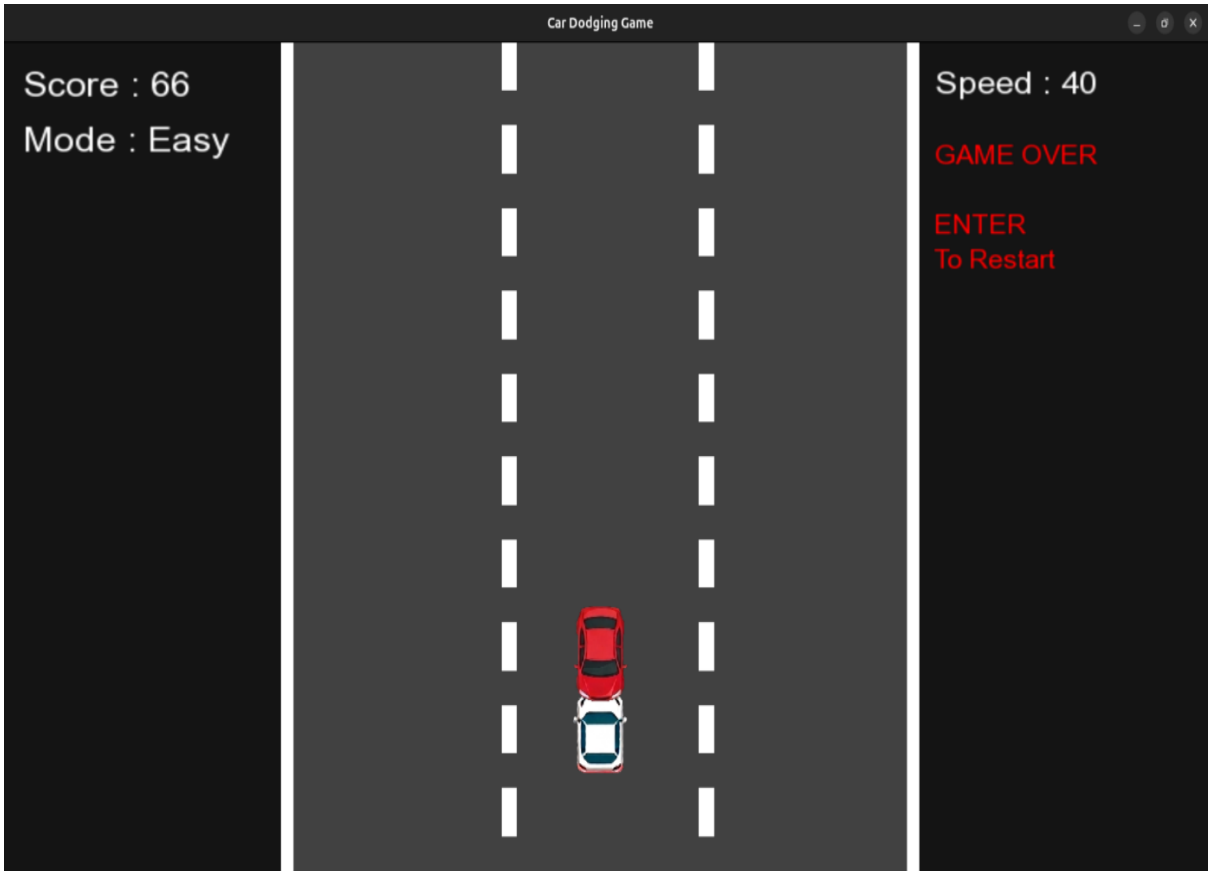


Figure 5: Collision Feature Working And Game Over Happened



07. Logical Understanding

The program flow for **Project-1** is organized around a continuous game execution cycle that manages user input, object movement, collision handling, score updates, and graphical rendering in real time. During each iteration of the loop, the system processes keyboard events, updates the positions of game objects, generates enemy vehicles, and redraws all visual elements on the screen to maintain smooth gameplay.

- The Game class manages the main game window, textures, sounds, score handling, enemy generation, collision checking, and overall game states.
- The Car class acts as a base class containing common properties such as sprite handling, positioning, rendering, and collision boundaries.
- The PlayerCar class extends the Car class and controls player movement across lanes while restricting movement within road limits.
- The EnemyCar class is responsible for enemy vehicle movement, lane allocation, off-screen removal, and score increment when enemies are avoided.
- The update mechanism handles player input, enemy movement, collision verification, difficulty progression, and timed enemy spawning.
- The rendering process displays the road background, player vehicle, enemy cars, score details, and additional screens such as pause menu or game-over interface.

The overall program flow demonstrates the practical implementation of real-time game loops, event-driven programming, collision management, and multimedia rendering using C++ and SFML. The project also improves understanding of software structure, object management, and interactive application development through a well-organized and modular game design approach.

08. Conclusion

In conclusion, this practical project was successfully completed using the C++ programming language and the SFML multimedia library. The development of the Car Dodger Game helped in strengthening the understanding of important programming concepts such as decision-making, iteration, functions, and object-oriented programming principles. The project provided practical exposure to real-time game development and demonstrated how programming logic and multimedia libraries can be combined to create an interactive and engaging gaming application. It also improved understanding of software structure, modular programming, and event-driven application design.

The Car Dodger Game was successfully designed and implemented with features such as smooth player movement, enemy vehicle generation, collision detection, score tracking, background movement, and increasing difficulty levels. The project helped in understanding important concepts including game loops, event handling, graphics rendering, sound integration, and efficient object management. Proper collision handling and optimized gameplay mechanics ensured stable performance, responsive controls, and smooth gameplay throughout the application. The use of multiple classes such as Car, PlayerCar, and EnemyCar also improved code reusability, maintainability, and overall software organization through the practical implementation of Object-Oriented Programming concepts.

Overall, the project successfully achieved all its intended objectives and provided valuable experience in developing multimedia applications using C++ and SFML. The development process improved practical coding skills, debugging techniques, logical thinking, and problem-solving abilities. Testing and debugging under different gameplay conditions helped in identifying errors and improving the reliability of the application. This project also increased confidence in software development and provided a strong foundation for future learning in the fields of game programming, graphics application development, and interactive software systems.

09. Future Scopes and Application

The future scope of this project includes several improvements that can enhance gameplay, graphics quality, and overall user experience. Although the current version of the Car Dodger Game successfully demonstrates the basic concepts of game development using C++ and SFML, many advanced features can still be added to make the game more interactive and challenging. The project provides a strong foundation for implementing additional gaming functionalities and improving real-time game mechanics.

Some important future improvements that can be added to this project include:

- Multiple levels with advanced difficulty modes for progressive gameplay challenges.
- Power-ups and bonus items such as speed boosts, shields, and extra lives
- Leaderboard system using file handling to maintain high scores and player rankings
- Multiplayer gameplay support for competitive gaming experiences
- Improved graphics, smoother animations, and enhanced sound effects
- Mobile platform support for Android and other portable devices
- AI-controlled enemy vehicles with intelligent movement patterns
- Different road themes and customizable player vehicles

The project can also be further extended into a complete arcade-style racing game with advanced gameplay features, mission-based objectives, and online score sharing systems. Additional improvements such as pause menus, settings options, and better visual effects can further increase the quality of the game. The concepts used in this project can also be applied in other multimedia and interactive software applications involving real-time graphics, event handling, and game development techniques. Overall, the project provides valuable practical experience in C++ programming, SFML library usage, software design, and real-time game development.

References

(1) Beginning C++ Game Programming by John Horton (Packt)

(2) Object Oriented Programming with C++ (The McGraw-Hill)

(3) SFML official documentation: https://www.sfml-dev.org/documentation/

Annexure

The following section presents the game source code along with explanations of the different components and functionalities implemented in the project.

Main.cpp:

Serves as the main entry point of the application and manages game initialization, window creation, asset loading, event handling, game loop execution, rendering, collision processing, scoring, difficulty management, and safe program termination.

```
#include <SFML/Graphics.hpp>
#include <SFML/Audio.hpp>
#include <vector>
#include <cstdlib>
#include <algorithm>
#include <string>
#include "PlayerCar.h"
#include "EnemyCar.h"
using namespace sf;

int main()
{
   // Create the game window
   RenderWindow window(VideoMode(800, 600), "Car Dodging Game");
   window.setFramerateLimit(60);

   // Load Textures
   Texture playerTex;

   Texture red1;
   Texture red2;
   Texture yellow1;
   Texture yellow2;
   Texture yellow3;

   // Load Player Car Texture
   if (!playerTex.loadFromFile("Assets/WhiteCar.png"))
   {
      return -1;
   }
   // Load Enemy Car Textures
   if (!red1.loadFromFile("Assets/RedCar1.png"))
   {
      return -1;
   }
   if (!red2.loadFromFile("Assets/RedCar2.png"))
```

```
{
   return -1;
}

if (!yellow1.loadFromFile("Assets/YellowCar1.png"))
{
   return -1;
}

if (!yellow2.loadFromFile("Assets/YellowCar2.png"))
{
   return -1;
}

if (!yellow3.loadFromFile("Assets/YellowCar3.png"))
{
   return -1;
}

// Store enemy textures
std::vector<Texture *> enemyTextures =
   {
      &red1,
      &red2,
      &yellow1,
      &yellow2,
      &yellow3};

// Create player object
PlayerCar player(playerTex);

// Set Initial Player Position
player.getSprite().setPosition(550, 500);

// Enemy vector
std::vector<EnemyCar> enemies;

// Score tracker
std::vector<bool> scored;

// Lane Positions
std::vector<float> lanes =
   {
      250,
      400,
      550};

Clock clock;
float spawnTimer = 0.0f;
float speedMultiplier = 1.0f;
bool gameOver = false;
```

```
bool paused = false;
bool gameStarted = false;

// Difficulty
int difficulty = 2;
float spawnDelay = 1.0f;
float speedGrowth = 0.05f;

// Load Font
Font font;

if (!font.loadFromFile("Assets/arial.ttf"))
{
   return -1;
}
int score = 0;
// UI Text Objects
Text scoreText;
Text speedText;
Text modeText;
Text gameOverText;
Text pauseText;
Text startText;

// Same Text Size
int uiSize = 24;

// Score Text
scoreText.setFont(font);
scoreText.setCharacterSize(uiSize);
scoreText.setPosition(20, 15);
scoreText.setFillColor(Color::White);

// Mode Text
modeText.setFont(font);
modeText.setCharacterSize(uiSize);
modeText.setPosition(20, 55);
modeText.setFillColor(Color::White);

// Speed Text
speedText.setFont(font);
speedText.setCharacterSize(22);
speedText.setPosition(620, 15);
speedText.setFillColor(Color::White);

// Pause Text
pauseText.setFont(font);
pauseText.setCharacterSize(18);
pauseText.setFillColor(Color::Yellow);

// PERFECTLY ALIGNED WITH SPEED TEXT
```

```
pauseText.setPosition(speedText.getPosition().x, 70);

// Better spacing
pauseText.setLineSpacing(1.2f);

pauseText.setString(
   "PAUSED\n"
   "Press R\n"
   "To Resume");

// Start Menu Text
startText.setFont(font);
startText.setCharacterSize(18);
startText.setFillColor(Color::White);

// PERFECTLY ALIGNED WITH SPEED TEXT
startText.setPosition(speedText.getPosition().x, 70);

// Better spacing
startText.setLineSpacing(1.2f);
startText.setString(
   "ENTER\n"
   "To Start\n\n"
   "1 : Easy\n"
   "2 : Medium\n"
   "3 : Hard");

// Game Over Text
gameOverText.setFont(font);
gameOverText.setCharacterSize(18);
gameOverText.setFillColor(Color::Red);

// PERFECTLY ALIGNED WITH SPEED TEXT
gameOverText.setPosition(speedText.getPosition().x, 70);

// Better spacing
gameOverText.setLineSpacing(1.2f);

gameOverText.setString(
   "GAME OVER\n\n"
   "ENTER\n"
   "To Restart");

// Load Sounds
SoundBuffer crashBuffer;
SoundBuffer moveBuffer;

// Load Crash Sound
if (!crashBuffer.loadFromFile("Assets/death.wav"))
{
   return -1;
```

```
}
// Load Move Sound
if (!moveBuffer.loadFromFile("Assets/chop.wav"))
{
   return -1;
}
Sound crashSound;
Sound moveSound;
crashSound.setBuffer(crashBuffer);
moveSound.setBuffer(moveBuffer);
// Background Music
Music bgMusic;

// Load Background Music
if (bgMusic.openFromFile("Assets/bgmusic.wav"))
{
   bgMusic.setLoop(true);
   bgMusic.setVolume(40.f);
   bgMusic.play();
}
// GAME LOOP
while (window.isOpen())
{
   float dt = clock.restart().asSeconds();

   Event event;

   // Event Handling
   while (window.pollEvent(event))
   {
      if (event.type == Event::Closed)
      {
         window.close();
      }

      if (event.type == Event::KeyPressed)
      {
         // Exit Game
         if (event.key.code == Keyboard::Escape)
         {
            window.close();
         }
         // Difficulty Selection
         if (!gameStarted)
         {
            if (event.key.code == Keyboard::Num1 ||
               event.key.code == Keyboard::Numpad1)
            {
               difficulty = 1;
            }
```

```
    if (event.key.code == Keyboard::Num2 ||
        event.key.code == Keyboard::Numpad2)
    {
        difficulty = 2;
    }
    if (event.key.code == Keyboard::Num3 ||
        event.key.code == Keyboard::Numpad3)
    {
        difficulty = 3;
    }
}
// Start Game
if (event.key.code == Keyboard::Enter &&
    !gameStarted)
{
    gameStarted = true;
}
// Restart Game
if (event.key.code == Keyboard::Enter &&
    gameOver)
{
    enemies.clear();
    scored.clear();
    score = 0;
    spawnTimer = 0;
    speedMultiplier = 1.0f;
    gameOver = false;
    paused = false;
    gameStarted = false;

    // RESET PLAYER COMPLETELY
    player.reset();
}

// Pause
if (event.key.code == Keyboard::P &&
    gameStarted &&
    !gameOver)
{
    paused = true;
}

// Resume
if (event.key.code == Keyboard::R &&
    gameStarted &&
    !gameOver)
{
    paused = false;
}
// PLAYER MOVEMENT FIXED
if (gameStarted &&
```

```
            !paused &&
            !gameOver)
        {
            Vector2f pos = player.getSprite().getPosition();
            // Move Left
            if (event.key.code == Keyboard::Left)
            {
                if (pos.x > 260)
                {
                    player.moveLeft();
                    moveSound.play();
                }
            }

            // Move Right
            if (event.key.code == Keyboard::Right)
            {
                if (pos.x < 540)
                {
                    player.moveRight();
                    moveSound.play();
}}}}}

// Game Updates
if (gameStarted &&
    !paused &&
    !gameOver)
{
    // Difficulty Settings
    if (difficulty == 1)
    {
        spawnDelay = 1.5f;
        speedGrowth = 0.03f;
    }
    else if (difficulty == 2)
    {
        spawnDelay = 1.0f;
        speedGrowth = 0.05f;
    }
    else
    {
        spawnDelay = 0.6f;
        speedGrowth = 0.08f;
    }

    // Increase Speed
    speedMultiplier += dt * speedGrowth;

    // Spawn Timer
    spawnTimer += dt;
```

```
// Spawn Enemy Cars
if (spawnTimer > spawnDelay)
{
   int lane = rand() % 3;
   int texIndex = rand() % enemyTextures.size();
   enemies.push_back(
      EnemyCar(
         *enemyTextures[texIndex],
         lanes[lane],
         lane));
   scored.push_back(false);
   spawnTimer = 0;
}

// Move Enemy Cars
for (auto &e : enemies)
{
   e.move(dt * speedMultiplier);
}

// Score Update
for (size_t i = 0; i < enemies.size(); i++)
{
   if (!scored[i] &&
      enemies[i].getSprite().getPosition().y >
         player.getSprite().getPosition().y)
   {
      score++;
      scored[i] = true;
   }
}

// Collision Detection
for (auto &e : enemies)
{
   if (player.getSprite()
         .getGlobalBounds()
         .intersects(
            e.getSprite()
               .getGlobalBounds()))
   {
      crashSound.play();
      gameOver = true;
      break;
   }
}

// Remove Offscreen Cars
for (size_t i = 0; i < enemies.size();)
{
   if (enemies[i].getSprite().getPosition().y > 700)
```

```
        {
            enemies.erase(enemies.begin() + i);
            scored.erase(scored.begin() + i);
        }
        else
        {  i++;  }
    }
}
// Update UI Text
scoreText.setString("Score : " + std::to_string(score));

speedText.setString("Speed : " + std::to_string((int)(speedMultiplier * 10)));
std::string diffName;

if (difficulty == 1)
{
    diffName = "Easy";
}
else if (difficulty == 2)
{
    diffName = "Medium";
}
else
{
    diffName = "Hard";
}
modeText.setString("Mode : " + diffName);

// Render
window.clear(Color(20, 20, 20));

// Road
RectangleShape road(Vector2f(420, 600));
road.setFillColor(Color(65, 65, 65));
road.setPosition(190, 0);
window.draw(road);

// Left Border
RectangleShape leftBorder(Vector2f(8, 600));
leftBorder.setFillColor(Color::White);
leftBorder.setPosition(190, 0);
window.draw(leftBorder);

// Right Border
RectangleShape rightBorder(Vector2f(8, 600));
rightBorder.setFillColor(Color::White);
rightBorder.setPosition(602, 0);
window.draw(rightBorder);

// Dotted Road Lines
for (int i = 0; i < 12; i++)
```

```
    {
        RectangleShape dash(Vector2f(10, 35));
        dash.setFillColor(Color::White);

        // Left Divider
        dash.setPosition(335, i * 60);
        window.draw(dash);

        // Right Divider
        dash.setPosition(
            465,
            i * 60);
        window.draw(dash);
    }

    // Draw Player
    if (gameStarted)
    {
        window.draw(player.getSprite());
    }

    // Draw Enemy Cars
    for (auto &e : enemies)
    {
        window.draw(e.getSprite());
    }
    // Draw UI
    window.draw(scoreText);
    window.draw(modeText);
    window.draw(speedText);

    // Draw Start Menu
    if (!gameStarted)
    {
        window.draw(startText);
    }

    // Draw Pause Text
    if (paused)
    {
        window.draw(pauseText);
    }
    // Draw Game Over Text
    if (gameOver)
    {
        window.draw(gameOverText);
    }
    window.display();
}
return 0; }
```

Car.h :

Defines the abstract base class for all car objects and provides common functionalities such as sprite handling, positioning, rendering support, and collision boundary access through shared sprite management.

```
#pragma once
#include <SFML/Graphics.hpp>
using namespace sf;

// Abstract Base Class for all cars
class Car
{
protected:
    Sprite sprite;

public:
    // Pure virtual function
    // Must be implemented in child classes
    virtual void move(float dt) = 0;

    // Return sprite reference
    Sprite &getSprite()
    {
        return sprite;
    }
};
```

PlayerCar.h:

Implements the player-controlled car class responsible for lane-based movement, keyboard interaction, initial positioning, sprite scaling, and restricting movement within predefined road boundaries using inherited features from the Car base class.

```
#pragma once
#include "Car.h"
#include <vector>

// Player Car Class
class PlayerCar : public Car
{

private:
    std::vector<float> lanes;
    int currentLane;
    float y;

public:
```

```
// Constructor
PlayerCar(Texture &texture)
{
   sprite.setTexture(texture);

   // Resize player car
   sprite.setScale(0.35f, 0.35f);

   // Set origin to center
   FloatRect bounds = sprite.getLocalBounds();

   sprite.setOrigin(bounds.width / 2, bounds.height / 2);

   // UPDATED ROAD LANES
   lanes = {250, 400, 550};
   currentLane = 2;
   y = 500;

   // Initial Position
   sprite.setPosition(lanes[currentLane], y);
}

// RESET PLAYER AFTER RESTART
void reset()
{
   currentLane = 2;
   sprite.setPosition(lanes[currentLane], y);
}
// Move to left lane
void moveLeft()
{
   if (currentLane > 0)
   {
      currentLane--;
      sprite.setPosition(lanes[currentLane], y);
   }
}
// Move to right lane
void moveRight()
{
   if (currentLane < 2)
   {
      currentLane++;
      sprite.setPosition(lanes[currentLane], y);
   }
}
void move(float dt) override
{  } };
```

EnemyCar.h:

Implements the enemy vehicle class responsible for downward movement, lane assignment, enemy spawning above the screen, and movement handling using inherited functionality from the Car base class to support collision detection and dodge-based scoring mechanics.

```
#pragma once
#include "Car.h"

// Enemy Car Class
class EnemyCar : public Car
{
private:
    float speed;
    int laneIndex;

public:
    // Constructor
    EnemyCar(Texture &texture, float x, int lane)
    {
        sprite.setTexture(texture);

        // Resize enemy car
        sprite.setScale(0.35f, 0.35f);

        // Set origin to center
        FloatRect bounds = sprite.getLocalBounds();

        sprite.setOrigin(bounds.width / 2, bounds.height / 2);

        // Spawn above screen
        sprite.setPosition(x, -100);

        speed = 200.0f;
        laneIndex = lane;
    }

    // Move enemy downward
    void move(float dt) override
    {
        sprite.move(0, speed * dt);
    }

    // Return lane index
    int getLane() const
    {
        return laneIndex;
    }
};
```